Programmation Orientée Objets

Chapitre 4
Héritage

Pr. Saïd OUATIK EL ALAOUI
s_ouatik@yahoo.com
ouatikelalaoui.said@uit.ac.ma

Plan

- **Notion d'Héritage**
- **Héritage simple**
  - Droits d'accès aux membres de la classe
  - Redéfinition de fonctions membres
  - Constructeurs, destructeurs
  - Amitié
- **Héritage multiple**
  - Syntaxe
  - Conflits
  - Héritage virtuel
- **Fonctions virtuelles**
- **Classes abstraites**

Notion d'Héritage

- Permet de définir une nouvelle classe (**classe dérivée**) à partir d'une ou plusieurs classes existantes (**classes de base**).

- La classe dérivée héritera les membres de la classe de base, tout en ajoutant de nouveaux attributs et de nouvelles méthodes

- Représente la relation **EST UN(E)**
  - Un cercle est une forme géométrique
  - Une voiture est un véhicule
  - Une employé est une personne

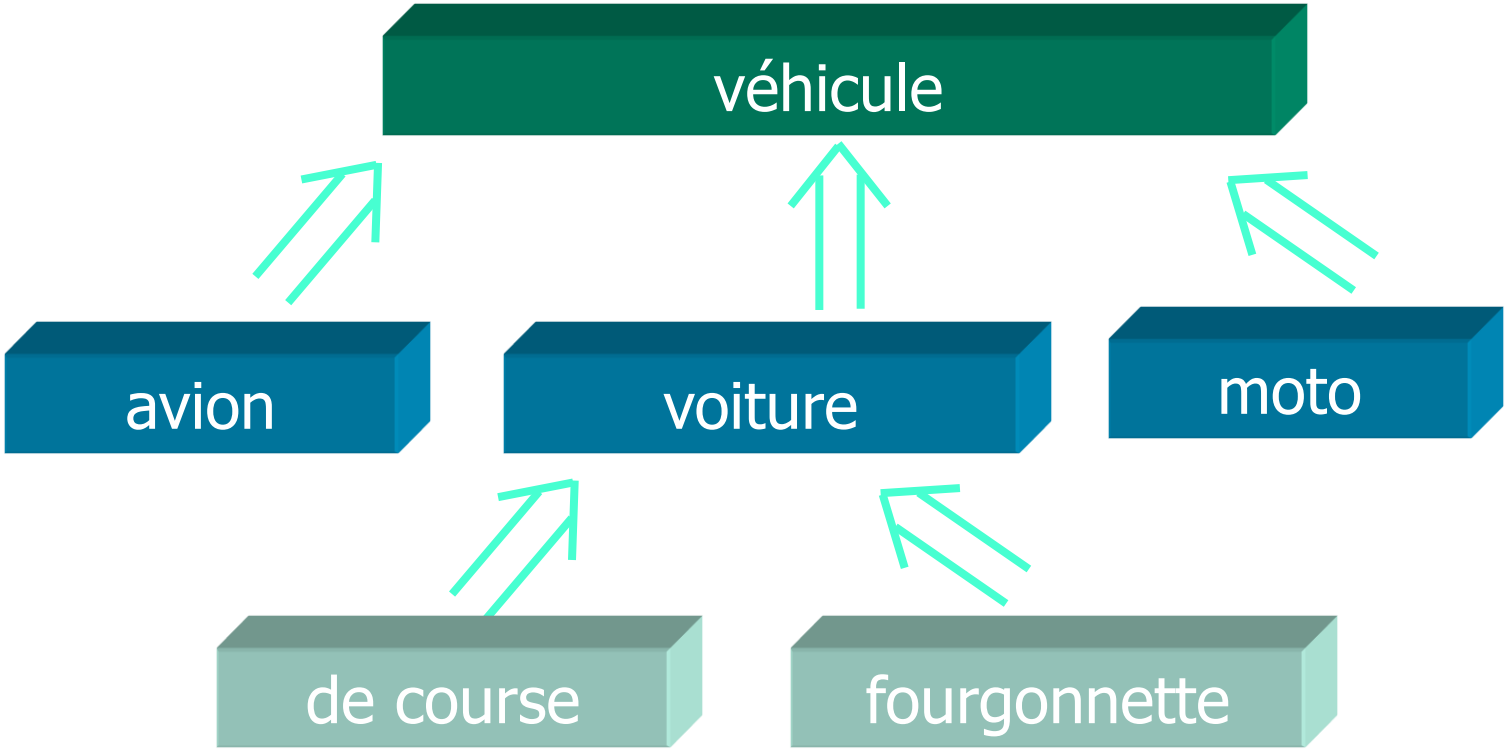
Notion d'Héritage

- L'héritage peut être:
  - **Simple** : Une seule classe de base
  - **Multiple**: Plusieurs classes de base

- Le concept d'héritage permet de:
  - **Exprimer** les relations structurelles et sémantiques entre les classes
  - **Réduire** les redondances de description et de stockage des propriétés
  - **Réutiliser** les classes existantes

Notion d'Héritage

Exemple



Notion d'Héritage

- La classe dérivée est un cas particulier de la classe de base
- Si une classe B dérive de la classe A, alors
  - **B** va **hériter implicitement** l'ensemble:
    - Des attributs de **A**
    - Des méthodes de **A**
  - **B** peut définir des attributs et des méthodes supplémentaires (**enrichissement**)
  - **B** Peut redéfinir certaines méthodes

Héritage simple

- *Syntaxe*

```
class class_derivee:public class_de_base
  { /* définitions des méthodes et des attributs
  spécifiques à la classe dérivée*/  …  };
```

- *Exemple*

```
class personne {
          char nom[20];
          char prenom[20];
          int  CIN;
        public:
          void saisir(char*,char*,int);
          void afficher();
          };
class etudiant : public personne {
                 int CNE;
           public:
                 void saisir_cne(int);
                 void afficher_cne();
          };
```

Héritage simple

Mode de dérivation

- ***Dérivation public:*** C'est le mode le plus utilisé
- **Syntaxe**

```
class class_derivee : public class_de_base
{ ----
 ---------
 public:
 --------
 ---------                    …
 };
```

Dans une dérivation publique, tous les membres public de la classe de base restent public dans la classe dérivée

Héritage simple
Mode de dérivation

- ***Remarques:***

  - **Les fonctions membres de la classe dérivée n'ont pas accès aux membres privés de la classe de base.**

  - **Dans une dérivation privée, l'utilisateur d'une classe dérivée n'a pas accès aux membres publique de la classe de base.**

```
class B : private A
{ ----
--------
public:
-------
--------
};
```

…

Héritage simple

Droit d'accès aux membres d'une classe

- Trois modes d'accès aux membres d'une classe:
  - *Public* : Accès permis à toute fonction à partir de n'importe quel point du programme
  - *private* **(**mode par défaut**)** : Accès permis aux fonctions membres de la classe
  - *protected* **(**lié à l'héritage**):** Accès permis aux fonctions membres de la classe et aux fonctions membres des classes dérivées .

Héritage simple

Droit d'accès aux membres de classe

- *Exemple*

```
class A {
     public:
       int pub;

     protected :
       int pro;

     private :
        int priv;
};

class B : public A {
     public:
       void fct(int x);
};
```

```
void B::fct(int x)
{
    pub = x    // ok accès possible
    pro = x;   // ok accès possible
    priv = x; //erreur
              //accès non autorisé
}

void fexterne(A pa, int x)
{
   pa.pro= x; // erreur
   pa.priv= x; // erreur
   pa.pub = x; // ok
}
```

Héritage simple

Redéfinition de fonctions

- Besoin d'adapter certaines fonctions de la classe de base à la classe dérivée
- La nouvelle fonction doit garder le même nom que celui de la classe de base
- Redéfinition ≠ surdéfinition
- La nouvelle définition (celle de la classe dérivée) va **masquer** l'ancienne (celle de la classe de base). Mais elle ne la remplace pas.

Héritage simple

Redéfinition de fonctions

- *Exemple*

```
class Rectangle {
    protected:
       float largeur;
       float hauteur;
    public:
       float surface();
};
float Rectangle::surface()
{  return largeur*hauteur;
}
class Rectangle3D:public Rectangle
   {
    protected:
       float profondeur;
   public:
       float surface();
};
```

```
float Rectangle3D::surface()
{
   return (2*largeur*hauteur+
   2*largeur*profondeur +
   2*hauteur*profondeur);
}

void main()
{
   Rectangle3D r;
   r.surface();//celle de la
   classe Rectangle3D
}
```

Héritage simple

Accès à une méthode de la classe base

- *Exemple*

```
class Rectangle3D :public Rectangle{
    protected:
        float profondeur;
   public:
       float surface();
};
float Rectangle3D::surface()
{ if (profondeur == 0)
        return Rectangle::surface();
  else
   return (2*largeur*hauteur+ 2*largeur*profondeur +
   2*hauteur*profondeur);
}
```

Héritage simple

Constructeur de la classe de base

- Trois cas possibles pour la classe de base
  - Constructeur n'est pas défini
    - Aucun problème ne se pose
  - Constructeur sans paramètres
    - Il sera appelé automatiquement à la création d'un objet de la classe dérivée.
  - Constructeur avec paramètres
    - Dans la définition du constructeur de la classe dérivée, on doit spécifier les paramètres à transmettre au constructeur de la classe de base.

Héritage simple

Constructeur de la classe de base

Exemple 1: Constructeur de la classe de base sans paramètres

```
Class Point
{ int x, y;
Public:
Point();
Void afficher();
};
Point:: Point()
{x=0;
Y=0;
}
Void Point :: affiche()
{
Cout<<"Cordonnees: "<<x<<y;
};
```

```
Class Point_colore: public Point
{int couleur;
Public:
Point_colore(int);
Void affiche();
};
Point_colore::Point_colore(int cl)
{couleur=cl;
}
Void Point_colore:: affiche()
{
Point::affiche(); // appel méthode de
                  //la classe de base
Cout<<« \nCouleur »<<couleur;
}
```

Héritage simple

Constructeur de la classe de base avec paramètres

- Dans ce cas, on doit définir un constructeur dans la classe dérivée.
- Comment passer les paramètres au constructeur de la classe de base?
  - *Explicitement:* lors de la définition du constructeur de la classe dérivée
  - *Syntaxe*

```
class_derivee::class_derivee(param1,param2,…):base(parami,…)
  {------
   ------
   --------
  }
```

- Exemple

Héritage simple

Constructeur de la classe de base

Exemple 2: Constructeur de la classe de base avec paramètres

```
class Point
{
protected:
int x, y;
Public:
Point(int, int);
void afficher();
};
Point:: Point(int abs,int ord)
{x=abs;
Y=ord;
}
void Point :: affiche()
{
cout<<"Cordonnees: "<<x<<y;
};
```

```
class Point_colore: public Point
{int couleur;
public:
Point_colore(int, int, int);
void affiche();
};
Point_colore::Point_colore(int abs,
int ord, int cl): Point(abs, ord)
{couleur=cl;
}
void Point_colore:: affiche()
{
cout<<x<<"    " <<y;
cout<<"\nCouleur"<<couleur;
}
```

Héritage simple
Constructeur de recopie

- Recopie des attributs de la classe de base A et ceux de la classe dérivée B

- Deux cas :
  - La classe dérivée n'a pas de constructeur de recopie
  - La classe dérivée possède un constructeur de recopie

Héritage simple
Constructeur de recopie

- Classe dérivée n'a pas de constructeur de recopie
  - Appel des constructeurs de recopie dans l'ordre suivant:
    - Constructeur de recopie explicite ou par défaut de la classe de base
    - Constructeur de recopie par défaut de la classe dérivée

Héritage simple
Constructeur de recopie

- Classe dérivée B possède un constructeur de recopie
  - Il sera appelé
  - Pas d'appel automatique du constructeur de recopie de la classe de base.
  - Le constructeur de recopie de A doit être mentionné dans l'entête du constructeur de recopie de B.
  - Sinon il y aura appel au constructeur sans arguments de A (obligatoire).

Héritage simple

Constructeur de recopie

Constructeur de recopie défini à la fois dans la classe de base et la classe dérivée

```
class Point
{ protected:
int x, y;
Public:
Point(int, int);
Point(Point &); //constr de recopie
Void afficher();
};
Point:: Point(int abs, int ord)
{x=abs;
y=ord;
}
void Point :: affiche()
{
Cout<<"Coordonnees: "<<x<<y;
};
```

```
class Point_colore: public Point
{int couleur;
Public:
Point_colore(int, int, int);
Point_colore(Point_colore &);
void affiche();
};
Point_colore::Point_colore(int abs,
int ord, int cl): Point(abs, ord)
{couleur=cl;
}
void Point_colore:: affiche()
{
Cout<<x<<<"    "<<y;
Cout<<"\nCouleur "<<couleur;
}
```

Héritage simple

Constructeur de recopie

Constructeur de recopie défini à la fois dans la classe de base et la classe dérivée

```
Point_colore::Point_colore(Point_colore & p): Point(p)
{
couleur=p.couleur;
}
```

Héritage simple
Constructeur/destructeur

- Ordre d'appel
  - Lors de la déclaration d'un objet de la classe dérivée, le constructeur de la classe de base est appelé **avant** celui de la classe dérivée
  - De façon symétrique le destructeur de la classe de base est appelée **après** le destructeur de la classe dérivée

Héritage simple
Amitié

- La déclaration d'amitié pour une classe accorde aux fonctions de cette classe les mêmes droits d'accès que ceux des fonctions membres
- La déclaration d'amitié pour une classe ne se propage pas aux classes dérivées
- L'amitié pour une fonction n'est pas héritée. A chaque dérivation il faut définir l'amitié pour cette fonction

Héritage multiple

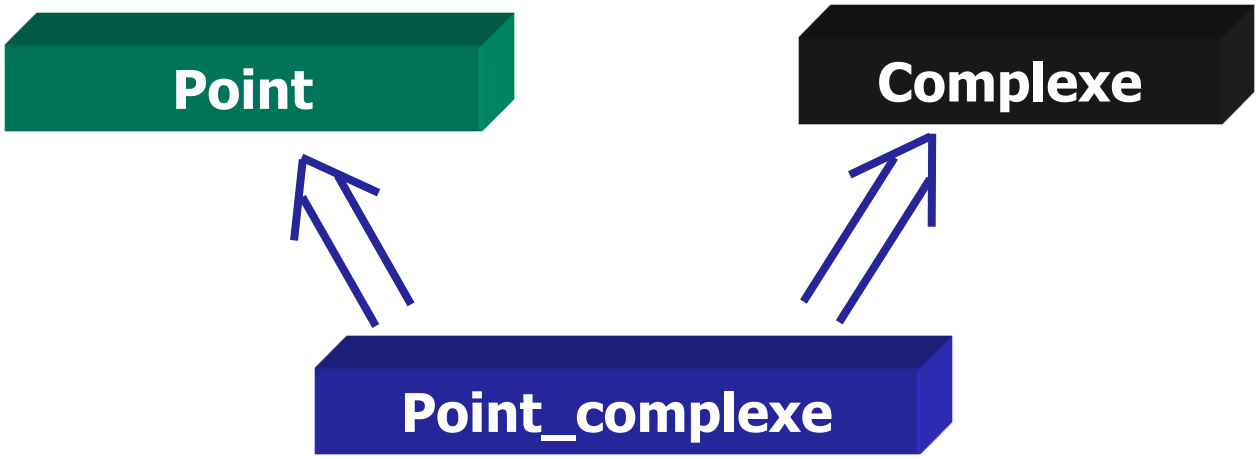
- *Syntaxe*

```
class D : mode B1, mode B2, … , mode Bn
{
….
}
```

  - Mode ={public, protected, private}

Héritage Multiple

Exemple



Héritage Multiple

Exemple -Classe point-

```
class point    {
        int x,y;
    public:
        void initialise(int,int);
        void deplace(int,int);
        void affiche_point();
                        };
void point::initialise(int abs,int ord)
{
        x=abs;
        y=ord;
}
void point::deplace(int dx,int dy)
{
        x+=dx;
        y+=dy;
}
void point::affiche_point()
{
        cout << "position :" <<x <<"," <<y <<"\n";
}
```

Héritage Multiple

Exemple -Classe complexe

```
class complexe          {
        float reel,img;
  public:
        void saisir(int,int);
        void affiche_complexe();
        };
void complexe::saisir(int r,int i)
{
        reel=r;
        img=i;
}
void complexe::affiche_complexe()
{
   cout << "abscisse :" <<x <<", ordonnee:" <<y <<"\n";
}
```

Héritage Multiple

Exemple -Classe point_complexe-

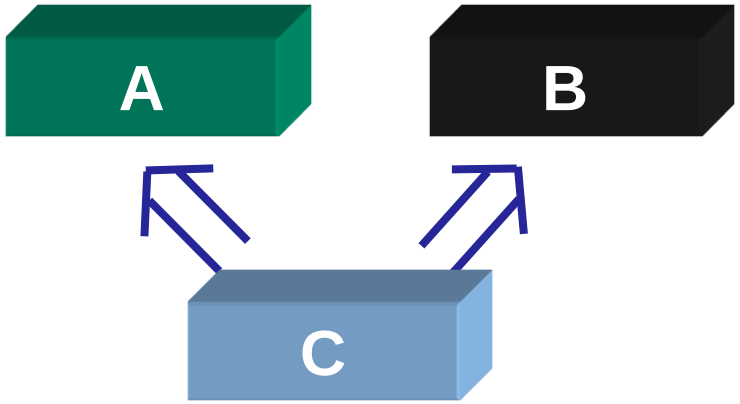
```
class point_complexe : public point, public complexe
{
      public:
            void initialiser_pc(int ,int );
            void affiche_pc();
};
void point_complexe::initialiser_pc(int n,int m)
{
initialise(n,m);
saisir(n,m);
}
void point_complexe::affiche_pc()
{
      affiche_point();
      affiche_complexe();
}
```

Héritage multiple

- Les constructeurs sont appelés dans l'ordre de déclaration des classes de base
- Les destructeurs sont appelés dans l'ordre inverse de celui des constructeurs
- Exemple

Héritage multiple

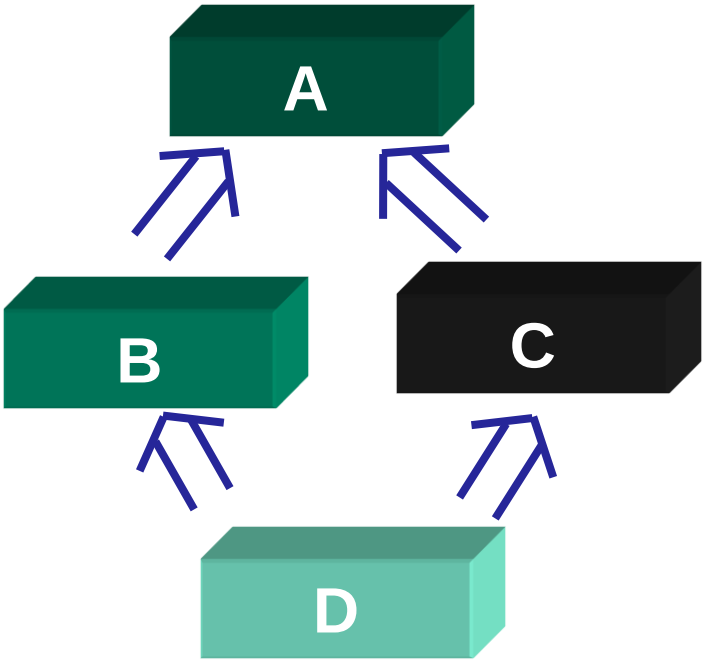
Conflits(1)



- *Problème*: Les classes A et B possèdent des attributs ou des fonctions portant le même nom
- *Solution*: Possibilité de les distinguer en utilisant l'opérateur de résolution **::**
- **Exemple**

Héritage multiple

Conflits(2)



- *Problème*: Duplication des membres fonctions ou attributs dans tous les objets de D
- *Solution* : Dérivation virtuelle : le membre sera présent en un seule exemplaire

Héritage virtuel

- Possibilité de déclarer une classe « virtuelle » pour indiquer au compilateur les classes à ne pas dupliquer
- Placement du mot clé virtuel devant le mode de dérivation de la classe
- *Syntaxe*

```
        class A {…}
// les descendants de B hériteront une seule fois
// les membres de A
        class B : public virtual A {…}
        class C : public virtual A {…}
        class D : public B, public C {…}
```

- Exemple

Héritage virtuel
Constructeurs

- Cas ou la classe A possède un constructeur avec paramètres
  - Pas de transmission d'arguments au constructeur de la classe A dans les classes B et C qui sont virtuels
  - Possibilité de transmettre les arguments nécessaires au constructeur de la classe A
  - Mais nécessité d'avoir un constructeur sans paramètres pour la classe A
- **Exemple**

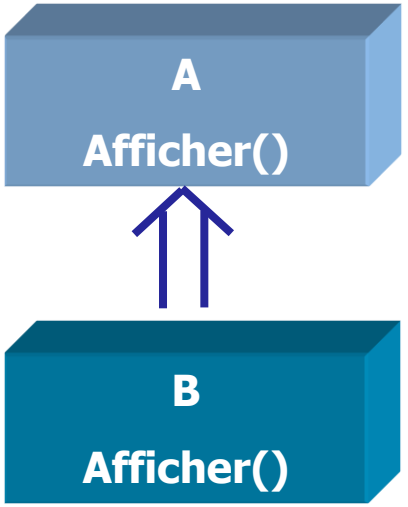
Fonctions virtuelles

notion de typage

- La notion de typage concerne les objets dynamiques et l'héritage
  - Soient les instructions suivantes:

```
A *pp;
B  b;
pp= &b;
```

    - pp : pointeur de type A
    - pp : pointe sur un objet de type B



  - L'instruction `pp->Afficher();` correspond à quelle fct afficher() de A ou de B?
  - **Question:** comment choisir la fonction à exécuter?

Fonctions virtuelles
notion de typage

- Typage statique:
  - Le type de l'objet pointé est déterminé au moment de la compilation
  - Le choix de la fonction est déterminé par le type du pointeur et non par celui de l'objet pointé
    - L'instruction `pp->Afficher();` correspond donc à celle de la classe A

Fonctions virtuelles
notion de typage

- Typage dynamique:
  - Le type de l'objet pointé est déterminé lors de l'exécution et non lors de la compilation
  - Le choix des fonctions est donc fait lors de l'exécution
  - Le choix de la fonction est déterminé par le type de l'objet pointé
    - L'instruction `pp->Afficher();` correspondra à celle de la classe B
  - Obtenu par les **fonctions virtuelles**

Fonctions virtuelles

- Faire précéder la fonction en question par le mot clé virtual

```
Class A
{protected:
……
public:
Virtual void fct();
….
};
```

```
Class B: public A
{……
public:
void fct();
….
};
```

- Le mot *virtual* est utilisé une seule fois. Il ne doit pas se répéter pour une même méthode au niveau des classes dérivées.
- Un constructeur ne peut pas être virtual, alors que le destructeur peut l'être.
- Ne pas confondre l'héritage virtuel et le statut virtuel des fonctions membres

Fonctions virtuelles pures

- Fonction déclarée
  - Comme méthode abstraite
  - Sans définition dans une classe (pas de code)
  - Déclarée avec initialisation à zéro
  - Devant être redéfinie dans les classes dérivées

- *Syntaxe*

```
virtual type_retour nom_fonc(param)=0;
```

Classes abstraites

- Une classe est dite abstraite s'il n'est pas possible de créer des objets de son type (non instanciable)
- Toute classe comportant une fonction virtuelle pure est abstraite
- Possibilité de définir des pointeurs et des références sur une classe abstraite
- Classe destinée uniquement à la dérivation
- *Exemple*

```
Class A
{protected:
……
public:
Virtual void affiche()=0;
….
};
```